

Materials Benchmark Authoring

From Paper PDF to Review-Ready Enhanced Harbor Tasks

STATUS	OWNER	LAST UPDATED
Proposed	MatBenchGuard / qa-review	August 3, 2026

Authors	Codex; MatBenchGuard project team
Reviewers	Independent Review Agent; materials-science reviewer
Related docs	Authoring, Review, and Repair Skills
Scope	PDF to paper-Gold-centered RESULT_ENHANCED Paper2Arm task.

1. Abstract

The system treats materials-science task authoring as an evidence-backed compilation pipeline. It freezes the PDF, uses UniParser to create paper/paper.md, builds a ledger for parameters, condition groups, Gold, and resources, and then enumerates several computational-science candidates. Only candidates that pass Q0, Gold applicability, resource closure, and affordable enhanced-checkpoint gates are compiled into Harbor packages.

The core guarantee is prove authorability before generating the package. Success is not a green local checker; it is PASS + RESULT_ENHANCED from an independent materials-benchmark-review. If no candidate qualifies, the system fails closed as NO_ENHANCED_CANDIDATE instead of deferring defects to large-scale Repair.

2. Goals and Non-Goals

Goals	Non-goals
Trace the paper question, method, parameters, Gold, and condition signature.	Do not invent unreported cutoff, mesh, timestep, model, or solver values.
Produce enhanced tasks with 60–80% Gold and 20–40% result checks.	Do not rerun DFT/MD/training or scan full large trajectories in the checker.
Express resources by stable filename and identity, independent of deployment paths.	Do not deploy resources to Bohrium/Playground or allocate Trisol IDs.
Publish only independently reviewed candidates; make failures replayable.	Do not create, inspect, run, hash, or cite solution/; do not use Repair as routine finishing.

3. Background and Problem Statement

material_v2_question contains 1,878 packages across 134 themes; the reviewed_passed comparison set contains 1,029 packages. Structural conventions are strong: ten core files are universal, nine instruction sections appear in about 98–100% of passed packages, outputs are mainly CSV/JSON, and the base image is almost uniform. However, resource methods, target policies, scoring tiers, paths, and bundle mappings have substantial historical drift.

Among 1,860 legacy final decisions, prominent hard-failure signals included 546 CHECKER_CORE_TASK_UNASSESSED cases, 142 unavailable indispensable inputs, and 100 invalid scientific targets. Current v3 Review is Baseline-first, but the distribution still argues for Q0 and asset closure before authoring, Gold freeze before checker generation, and a real low-cost result checkpoint during candidate selection.

4. Proposed Architecture

The pipeline separates scientific evidence, candidate selection, package compilation, and independent acceptance.

Materials Benchmark Authoring/Evidence-backed compilation pipeline

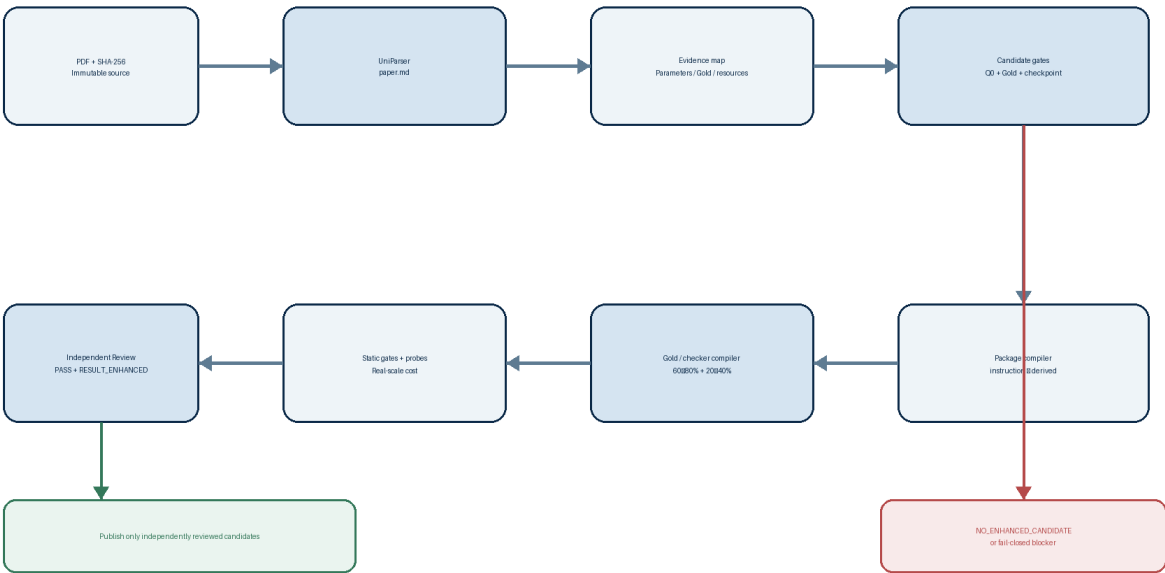


Figure 1. Evidence-backed Authoring compilation pipeline. Red is a fail-closed authoring exit; green accepts only independently reviewed enhanced candidates.

Core components

Component	Responsibility	Primary storage	Failure behavior
Source Freezer + UniParser Adapter	Freeze PDF hash; choose digital/OCR flags; create paper.md; record token and parse quality.	processing/source + authoring_record.json	BLOCKED_SOURCE_PARSE if load-bearing equations, tables, or captions are missing.
Evidence Mapper	Extract system, method, four parameter classes, condition groups, Gold candidates, resources, and workflow edges.	external authoring_record.json	Do not enter candidate selection while a core claim lacks a locator.
Candidate Gate & Selector	Enumerate at least three candidates when possible; gate lexicographically on Q0, Gold, resources, workflow, and checkpoint.	candidate_records	NO_ENHANCED_CANDIDATE if all fail; emit no weak package.
Package + Gold/Checker Compiler	Write instruction first, synchronize JSON/TOML/tests, and freeze Gold, tolerance, weights, and hidden references.	processing/candidate + evidence	Return to evidence mapping on condition mismatch, path coupling, or guessed values.
Validators + Independent Review Bridge	Run schema, probe, and cost gates; use independent Review to decide publishability.	validation logs + independent_review	Local pass is not scientific approval; Review failure returns to Authoring.

5. Request Lifecycle

1. Accept the PDF, stable paper_id, task name, and optional resources; create an external processing workspace.
2. Compute PDF SHA-256; use UniParser digital or OCR mode; inspect equations, tables, captions, and units.
3. Build the paper evidence map; classify parameters as PAPER_FIXED, SOLVER_SEARCHABLE, TARGET_DEFINING, or INDISPENSABLE_ASSET.
4. Enumerate candidates and run Q0 first; bind each to condition groups, Gold, resources, workflow, and checkpoint.
5. Select only an enhanced-feasible candidate; write a self-contained instruction, then derive steps/manifest/resources/task.
6. Freeze Gold and tolerance; generate a 60–80% Gold plus 20–40% result-check checker; run Baseline, risk-matched, and real-scale cost probes.
7. Run authoring/package/Harbor static validation and independent Review; publish only PASS + RESULT_ENHANCED.

6. API and Data Contracts

Primary data contract: authoring_record.json

Field	Type	Required	Description
source	object	Yes	PDF path/hash, UniParser token/flags, and paper.md path; never the API key.
parse_quality	object	Yes	Text/equation/table/caption completeness and blockers.
candidate_records	array	Yes	Q0, Gold, resources, checkpoints, and SELECTED/REJECTED decision for each candidate.
parameter_records	array	Yes	Four parameter classes plus three unique-value booleans that expose hidden fixation.
condition_group_records + gold_records	arrays	Yes	Complete signatures, target mapping, policy, applicability, and provenance.
enhancement + probe_records	objects	Yes	Gold/result weights, result checks, five Baseline probes, and at least one enhanced probe.
checker_cost_record + independent_review	objects	Yes	Real-scale cost and final verdict/quality_tier/publishable; validator runs the complete materials-core-review/3.3 artifact and package-aware consistency checks.

Contract guarantees

The authoring record is the single external source of truth for authoring decisions; the Harbor candidate stores only solver/reviewer package files. Publish validation runs the complete materials-core-review/3.3 validator and its package-aware tier, weight, and tolerance checks; legacy lowercase scoring_tier cannot bypass this gate.

- instruction.md precedes all derived views; tests cannot add public requirements.
- Every target, condition, resource, workflow, and output ID is unique and closed; resource records, resources.json, mapping types, bundled filenames, and files must agree.
- Bundled authoring files live at resources/<filename>; instruction uses filename only; deployment locator is late-bound. Output paths are canonical below /app/outputs; paper_id cannot be a path or the reserved solution name.
- Source hash, parse flags, Gold provenance, tolerance boundary, and the complete external Review artifact support audit and replay; four-field Review stubs are rejected.

Schema and implementation:

.cursor/skills/materials-benchmark-authoring/: assets/authoring_record_template.json and scripts/validate_authoring_record.py.

7. Consistency, Idempotency, and Replay

Concurrent work is isolated by authoring_id; the design adds no global lock or dispatcher. Human prompts coordinate publishing and corpus_review_tracking.json updates. Source PDF, evidence, and candidate are never silently overwritten to manufacture a latest version.

Scenario	Expected behavior	Reasoning
Same PDF submitted again	Reuse a complete parse for the same hash; issue a new authoring_id for a different hash.	Prevents stale paper.md from being paired with a new PDF.

Scenario	Expected behavior	Reasoning
Parse, Gold, or resource write fails	Emit no publish candidate; record blocker and last successful stage.	Fail closed so partial evidence is never treated as complete.
Checker probe, cost, or Review fails	Keep candidate and evidence; revise from the corresponding upstream layer; do not publish.	Keeps the repair loop inside the authoring workspace.
Rules or Harbor schema change	Capture Skill/Review versions; rerun static validation and independent Review.	Historical PASS does not automatically remain publishable.

8. Security and Privacy Considerations

- Read UNIPARSER_API_KEY from the environment only; never store it in the Skill, record, logs, or package.
- Do not record Authorization headers, pre-signed URLs, or reusable credentials; retain only task token and non-sensitive flags.
- Paper PDF, parsed Markdown, hidden Gold, and references may have licensing constraints; check access, distribution, and retention separately.
- Hidden Gold, exact tolerances, and checker implementation stay out of instruction; submission requirements remain fully public.
- Any solution path component is excluded recursively from Authoring, Review, and Repair; Harbor scaffolding must use --no-solution.

9. Operational Readiness

Signal	SLO or alert	Owner	Launch gate
Review first-pass enhanced rate	Pilot target at least 80%; every failure has a cited gate and evidence.	Authoring + independent Review	Required
Gold/condition/resource closure	100% closed at review-ready; any dangling ID blocks.	Authoring validator	Required
Checker correctness probes	Five Baseline plus at least one enhanced probe pass; retain per-component reward.	Checker author	Required
Checker cost	At most 32 CPU or 1 H100 and 600 s; real-scale; no primary recomputation.	Review operational gate	Required
UniParser quality	Load-bearing text/equation/table/caption all PASS; otherwise reparse or block.	Source/parse owner	Required

10. Alternatives Considered

Alternative	Why it was considered	Why it was not selected
One-shot LLM: PDF to full package	Fast and low-call-count.	Gold, conditions, resources, and checker are generated together, so errors can self-certify and are hard to localize.
Author Baseline, then route everything to Repair	Reuses existing Repair capability.	Contradicts the goal of reducing Repair and exposes defects only after package generation.
Hard-code /app/resources paths in instruction	Direct in-container convention.	Couples the scientific prompt to Playground, Bohrium, and local mount details.
Rerun the full primary computation in checker	Appears hardest to game.	Exceeds cost budgets; prefer result invariants and offline references over duplicate solving.

11. Open Questions

1. Should new tasks publish into material_v3_question or a new authoring-generated corpus? Recommendation: keep them in independent processing until a human decides.
2. Who owns resource_unique_key to Playground/Trisol ID deployment mapping? Recommendation: authors declare stable identity; deployers add locator.
3. When should the Paper2Arm schema 1.3 profile migrate to Harbor 1.4? Recommendation: retain the corpus profile and use harbor check for drift detection.
4. Figure-digitized Gold needs a separate policy for allowed tools, dual review, and uncertainty recording; until then unauditable figure values cannot be PAPER_DIRECT.

12. Decision and Next Steps

Adopt the evidence-backed compiler architecture and use ``.cursor/skills/materials-benchmark-authoring/`` as the M1 implementation. The initial version includes the workflow, references, solution-free template, authoring record, package validator, and positive/negative contract tests. Publish requires a complete Review 3.3 artifact, package-aware consistency, canonical output paths, closed resources, and a checker within the CPU/single-H100 budget. The next priority is forward-testing real papers across task types, measuring first-pass Review performance, and tightening gates exposed by failures.

Milestone	Deliverable	Exit criteria
M1	Authoring Skill, schemas, solution-free scaffold, static validators, and System Design.	Skill validator plus negative draft and positive review-ready smoke tests pass.
M2	Forward-test 3–5 real PDFs across DFT, MD, and model fitting.	Each evidence ledger closes; independent Review failures are reproducible and produce gate revisions.
M3	Stabilize resource deployment mapping and independent Review handoff.	Filename-only instruction works locally and in Playground without prompt edits.
M4	Batch Authoring pilot and first-pass metrics dashboard.	Enhanced first-pass target is met; Repair demand falls without Gold or resource regressions.